

Реализована простая модель сверточного автоэнкодера для реконструкции черно-белых изображений.

В качестве входных данных были использованы изображения из датасета "A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection"(Paul Bergmann, Michael Fauser, David Sattlegger, and Carsten Steger, IEEE Conference on Computer Vision and Pattern Recognition, 2019). Для упрощения расчетов разрешение изображений было понижено до [128 128] (исходное разрешение - [1024 1024]). Изображения нормированы в диапазон [0, 1]. Модель обучалась на примерах «нормальных» данных (без дефектов).

Модель реализована на языке MATLAB с использованием пакета "Deep Learning Toolbox".

Модель представляет собой сверточный автоэнкодер с пятью сверточными слоями в кодере и пятью зеркальными повышающими свертками в декодере. Повышающие свертки в декодере реализованы в виде комбинации слоев `resize2dLayer` с билинейной интерполяцией и `convolution2dLayer`. После каждого слоя свертки был поставлен слой `maxPooling` с ядром размера 2.

В качестве функции активации была использована функция `leakyRelu` (`scale = 0.01`). Целевой функцией было выбрано среднеквадратичное отклонение (`mse(target, item)`) между исходным изображением и реконструированным из латентного пространства. Размерность латентного пространства: 128.

Параметры обучения модели:

- Количество изображений для обучения: 320
- Количество эпох обучения: 80
- Learning rate: 0.001

Сравнительный анализ работы модели при различных значениях гиперпараметров проведен не был.

Схема архитектуры модели приведена на рис. 1, 2

Analysis for dlnetwork usage

Name: netE

Analysis date: 10-Dec-2025 09:07:22



LAYER INFORMATION				
	Name	Type	Activations S (Spatial), T (Time), C (Channel), B (...)	Learnable Sizes
1	imageinput 128x128x1 images	Image Input	128(S) × 128(S) × 1(C) × 1(B)	-
2	conv_1 8 3x3 convolutions with stride [1 1] and ...	2-D Convolution	128(S) × 128(S) × 8(C) × 1(B)	Weights: 3 × 3 × 1 - Bias: 1 × 1 × 8
3	E activation_1_1 Leaky ReLU with scale 0.01	Leaky ReLU	128(S) × 128(S) × 8(C) × 1(B)	-
4	maxpool_1 2x2 max pooling with stride [2 2] and pa...	2-D Max Pooling	64(S) × 64(S) × 8(C) × 1(B)	-
5	conv_2 16 3x3 convolutions with stride [1 1] and ...	2-D Convolution	64(S) × 64(S) × 16(C) × 1(B)	Weights: 3 × 3 × 8 - Bias: 1 × 1 × 16
6	E activation_2_1 Leaky ReLU with scale 0.01	Leaky ReLU	64(S) × 64(S) × 16(C) × 1(B)	-
7	maxpool_2 2x2 max pooling with stride [2 2] and pa...	2-D Max Pooling	32(S) × 32(S) × 16(C) × 1(B)	-
8	conv_3 32 3x3 convolutions with stride [1 1] and ...	2-D Convolution	32(S) × 32(S) × 32(C) × 1(B)	Weights: 3 × 3 × 16 - Bias: 1 × 1 × 32
9	E activation_3_1 Leaky ReLU with scale 0.01	Leaky ReLU	32(S) × 32(S) × 32(C) × 1(B)	-
10	maxpool_3 2x2 max pooling with stride [2 2] and pa...	2-D Max Pooling	16(S) × 16(S) × 32(C) × 1(B)	-
11	conv_4 64 3x3 convolutions with stride [1 1] and ...	2-D Convolution	16(S) × 16(S) × 64(C) × 1(B)	Weights: 3 × 3 × 32 - Bias: 1 × 1 × 64
12	E activation_4_1 Leaky ReLU with scale 0.01	Leaky ReLU	16(S) × 16(S) × 64(C) × 1(B)	-
13	maxpool_4 2x2 max pooling with stride [2 2] and pa...	2-D Max Pooling	8(S) × 8(S) × 64(C) × 1(B)	-
14	conv_5 128 3x3 convolutions with stride [1 1] an...	2-D Convolution	8(S) × 8(S) × 128(C) × 1(B)	Weights: 3 × 3 × 64 - Bias: 1 × 1 × 128
15	E activation_5_1 Leaky ReLU with scale 0.01	Leaky ReLU	8(S) × 8(S) × 128(C) × 1(B)	-
16	maxpool_5 2x2 max pooling with stride [2 2] and pa...	2-D Max Pooling	4(S) × 4(S) × 128(C) × 1(B)	-
17	flatten Flatten	Flatten	2048(C) × 1(B)	-
18	fc 128 fully connected layer	Fully Connected	128(C) × 1(B)	Weights: 128 × 2048 Bias: 128 × 1

Рис. 1: Архитектура кодера

Analysis for dlnetwork usage

Name: netD

Analysis date: 10-Dec-2025 09:07:30



LAYER INFORMATION				
	Name	Type	Activations S (Spatial), T (Time), C (Channel), B (...)	Learnable Sizes
1	input 128 Features	Feature Input	128(C) × 1(B)	-
2	fc 2048 fully connected layer	Fully Connected	2048(C) × 1(B)	Weights: 2048 × 128 Bias: 2048 × 1
3	reshape Reshape to size [4 4] in "S" (spatial) an...	Reshape	4(S) × 4(S) × 128(C) × 1(B)	-
4	layer_1 Resize 2d layer with scale of [2 2]	Resize	8(S) × 8(S) × 128(C) × 1(B)	-
5	conv_1 128 3x3x128 convolutions with stride [1 ...	2-D Convolution	8(S) × 8(S) × 128(C) × 1(B)	Weights: 3 × 3 × 128 - Bias: 1 × 1 × 128
6	D activation_1_1 Leaky ReLU with scale 0.01	Leaky ReLU	8(S) × 8(S) × 128(C) × 1(B)	-
7	layer_2 Resize 2d layer with scale of [2 2]	Resize	16(S) × 16(S) × 128(C) × 1(B)	-
8	conv_2 64 3x3x128 convolutions with stride [1 ...	2-D Convolution	16(S) × 16(S) × 64(C) × 1(B)	Weights: 3 × 3 × 128 - Bias: 1 × 1 × 64
9	D activation_2_1 Leaky ReLU with scale 0.01	Leaky ReLU	16(S) × 16(S) × 64(C) × 1(B)	-
10	layer_3 Resize 2d layer with scale of [2 2]	Resize	32(S) × 32(S) × 64(C) × 1(B)	-
11	conv_3 32 3x3x64 convolutions with stride [1 ...	2-D Convolution	32(S) × 32(S) × 32(C) × 1(B)	Weights: 3 × 3 × 64 - Bias: 1 × 1 × 32
12	D activation_3_1 Leaky ReLU with scale 0.01	Leaky ReLU	32(S) × 32(S) × 32(C) × 1(B)	-
13	layer_4 Resize 2d layer with scale of [2 2]	Resize	64(S) × 64(S) × 32(C) × 1(B)	-
14	conv_4 16 3x3x32 convolutions with stride [1 ...	2-D Convolution	64(S) × 64(S) × 16(C) × 1(B)	Weights: 3 × 3 × 32 - Bias: 1 × 1 × 16
15	D activation_4_1 Leaky ReLU with scale 0.01	Leaky ReLU	64(S) × 64(S) × 16(C) × 1(B)	-
16	layer_5 Resize 2d layer with scale of [2 2]	Resize	128(S) × 128(S) × 16(C) × 1(B)	-
17	conv_5 8 3x3x16 convolutions with stride [1 ...]	2-D Convolution	128(S) × 128(S) × 8(C) × 1(B)	Weights: 3 × 3 × 16 - Bias: 1 × 1 × 8
18	D activation_5_1 Leaky ReLU with scale 0.01	Leaky ReLU	128(S) × 128(S) × 8(C) × 1(B)	-
19	transposed-conv 1 3x3x8 transposed convolutions with st...	2-D Transposed Con...	128(S) × 128(S) × 1(C) × 1(B)	Weights: 3 × 3 × 1 - Bias: 1 × 1 × 1
20	layer_6 sigmoid	Sigmoid	128(S) × 128(S) × 1(C) × 1(B)	-

Рис. 2: Архитектура декодера

1 Результаты

Время обучения модели: 29 минут 31 секунда.

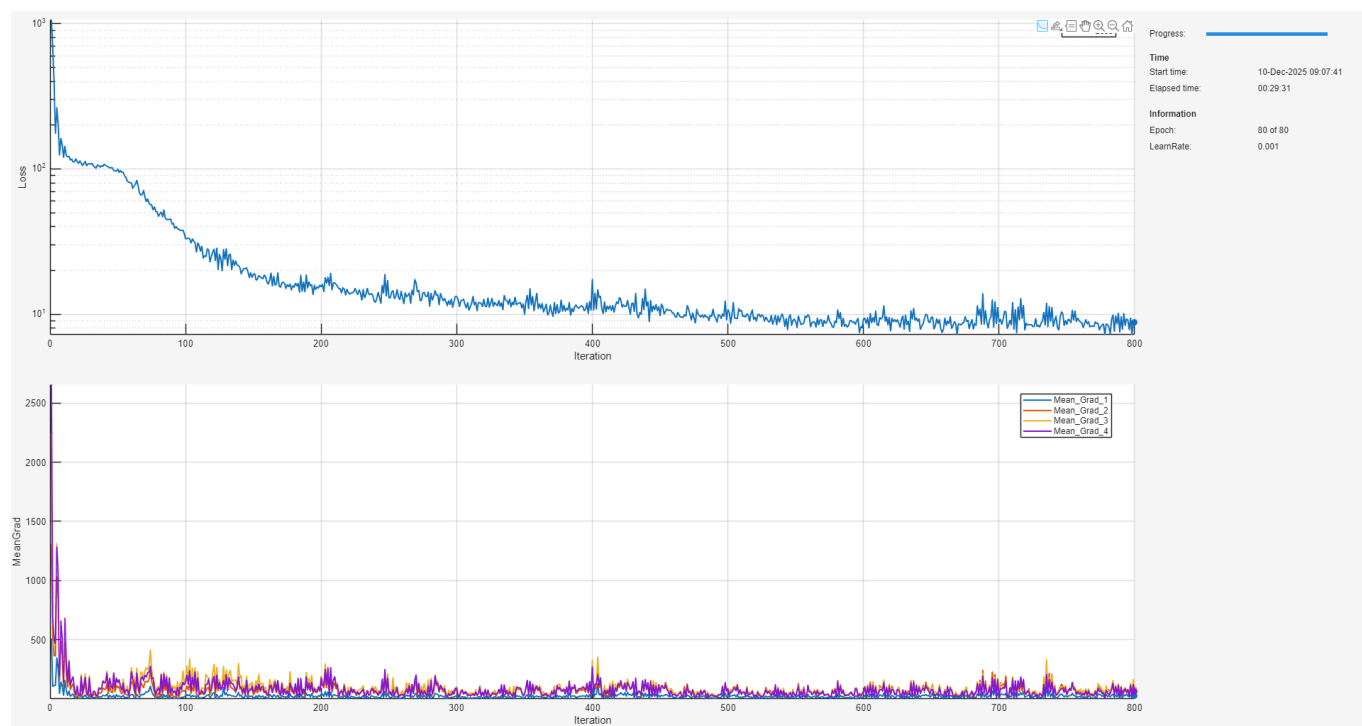


Рис. 3: Прогресс обучения

Реконструированные изображения

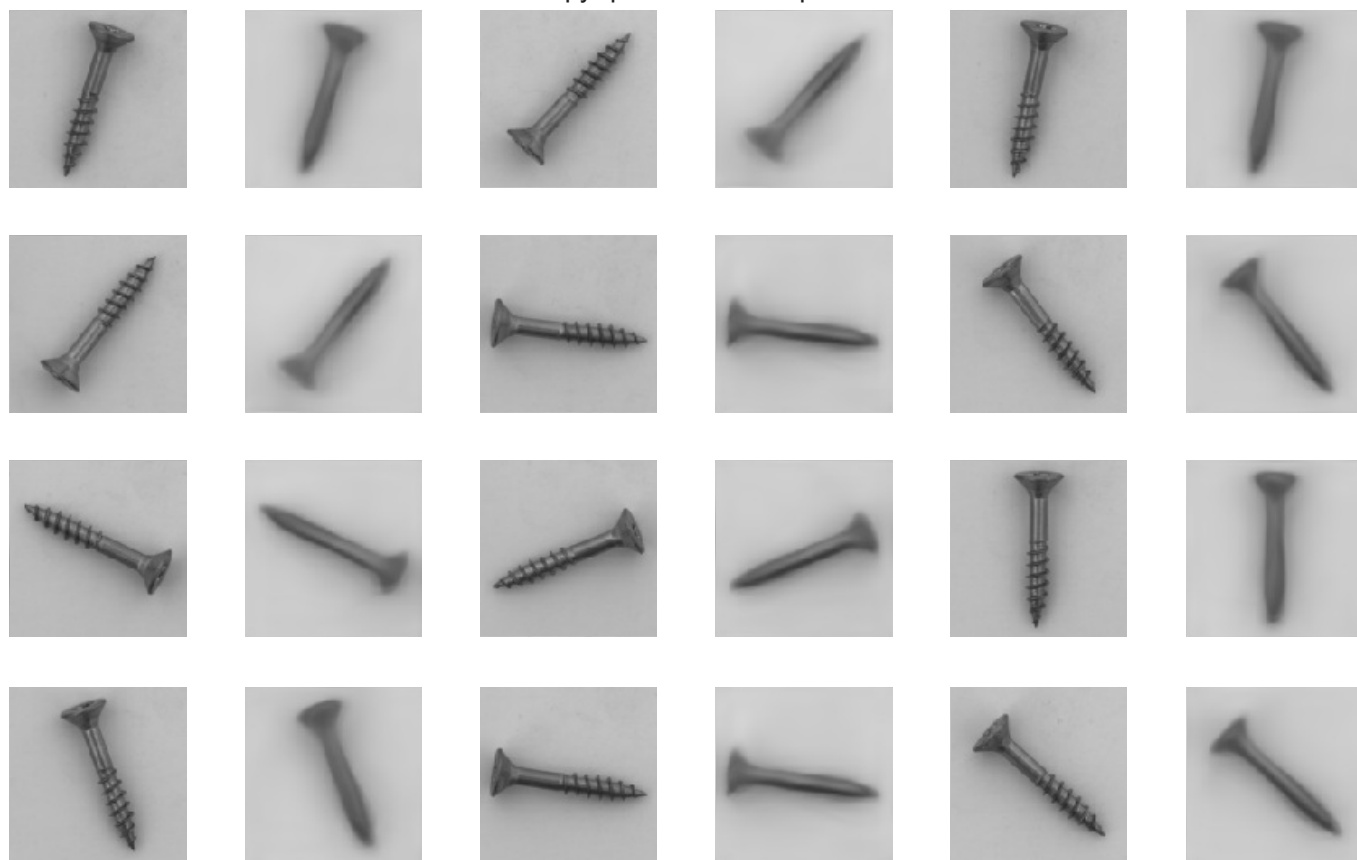


Рис. 4: Реконструированные изображения

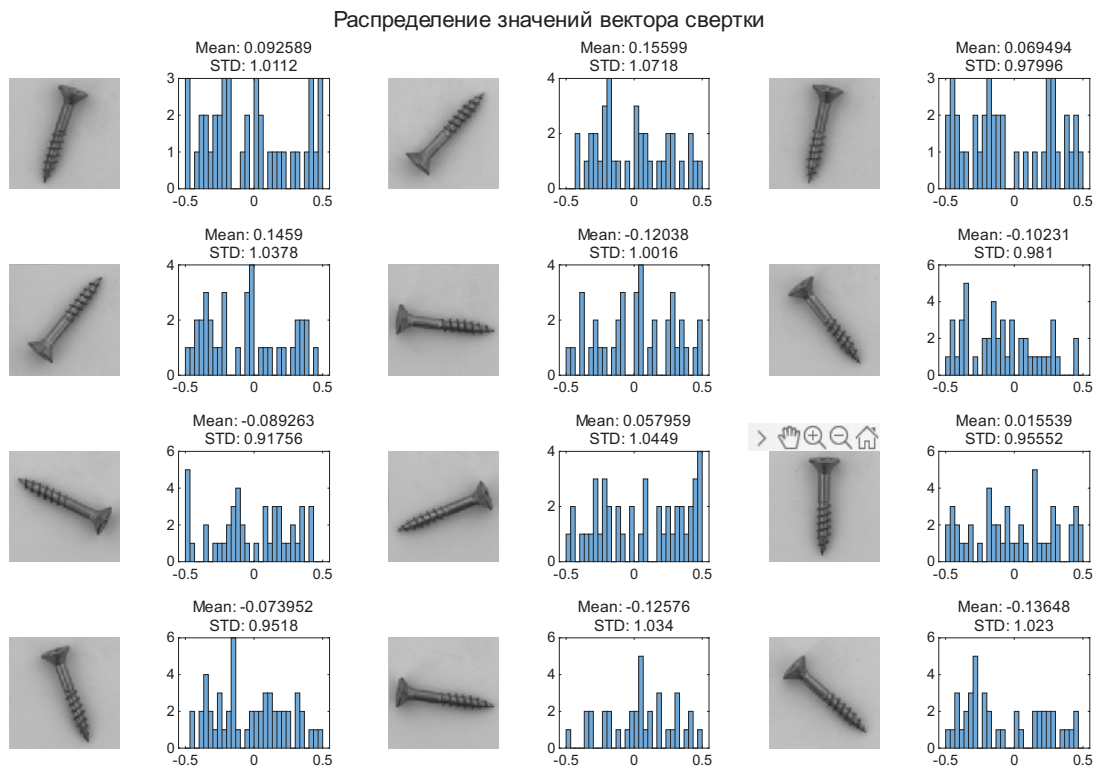


Рис. 5



Рис. 6: Пример карты ошибок

На рис. 4 видно, что реконструированные изображения сохраняют ориентацию исходного изображения. Также улавливается направление освещения. Информация о винтовой части детали в процессе обучения теряется. На рис. 5 изображено распределение значений латентного вектора модели. Очевидных связей с какими-либо признаками исходных изображений не обнаружено. Вопрос требует дополнительных исследований.

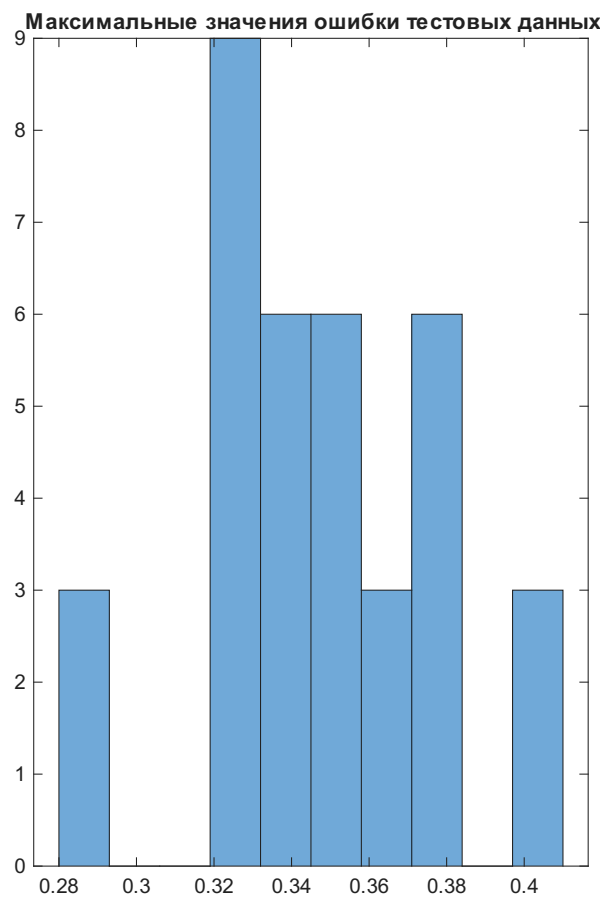
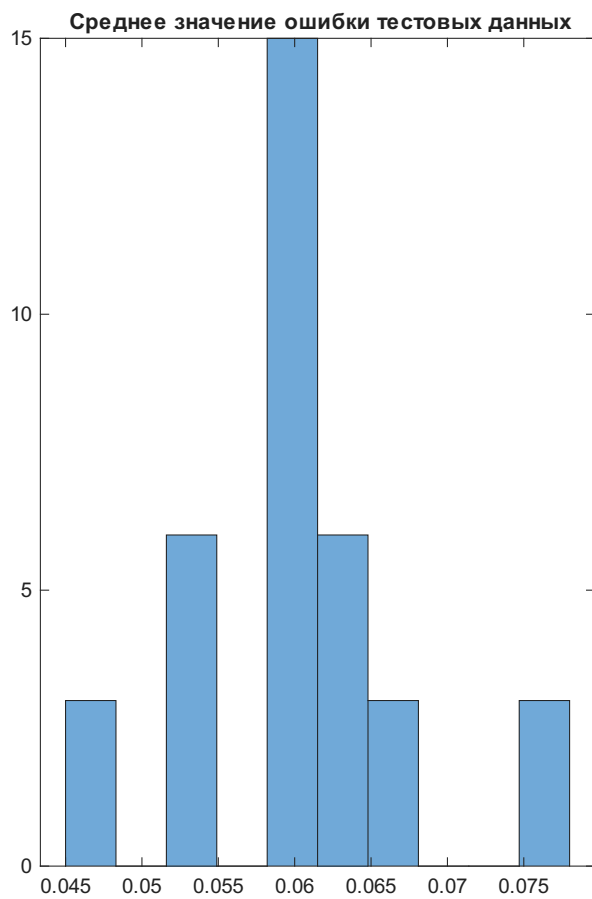


Рис. 7

Средние ошибки реконструкции тестовых данных лежат в диапазоне между 0.06 - 0.065. Однако максимальные ошибки реконструкции довольно значительны: нижняя граница на уровне 0.28, верхняя - 0.4.

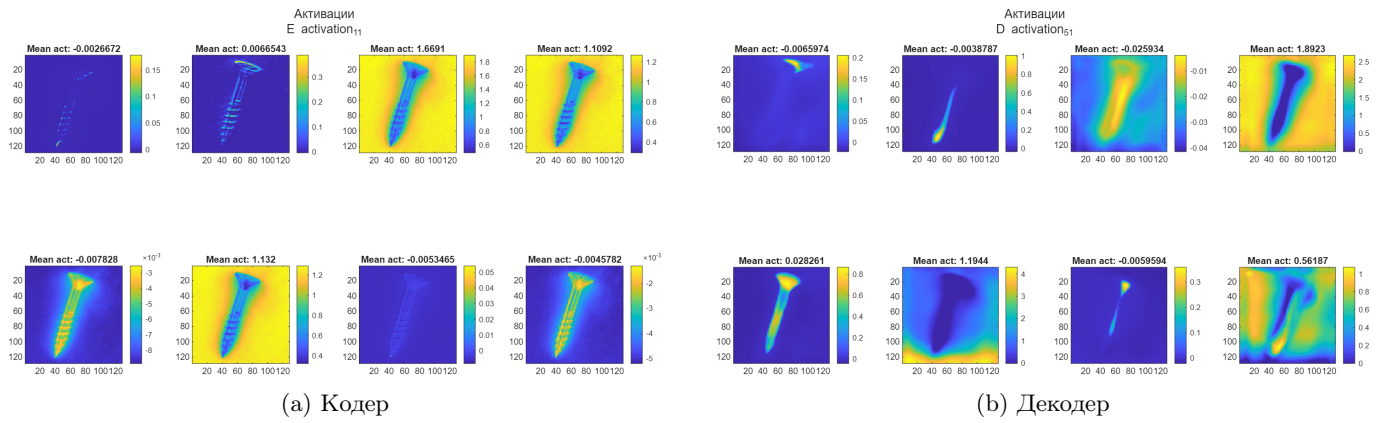


Рис. 8: Активации первого слоя Кодера и последнего слоя декодера

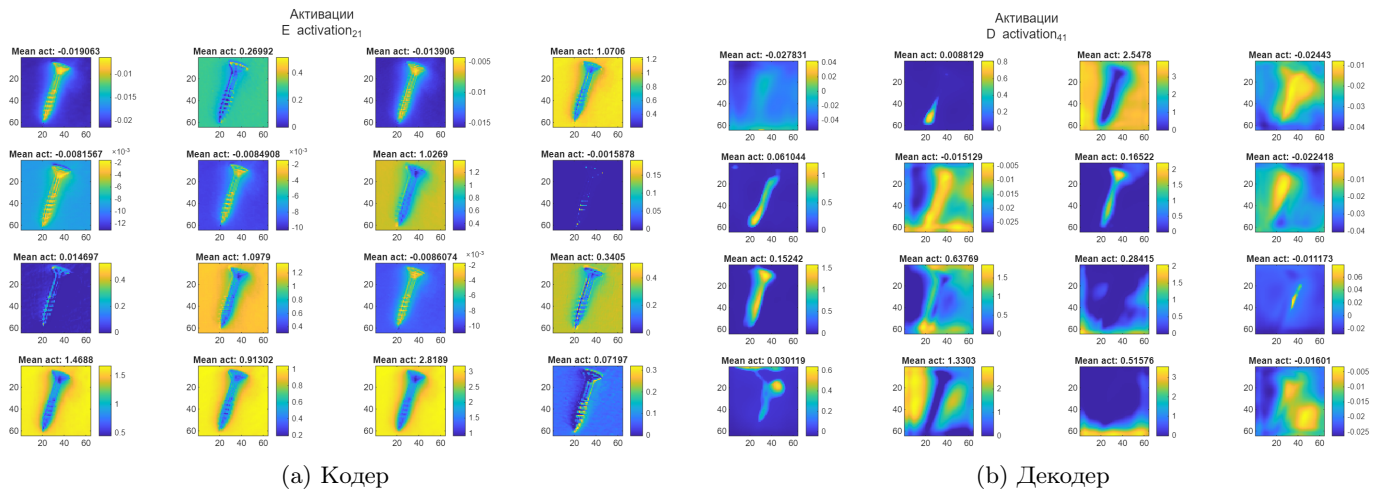


Рис. 9: Активации второго слоя Кодера и четвертого слоя декодера

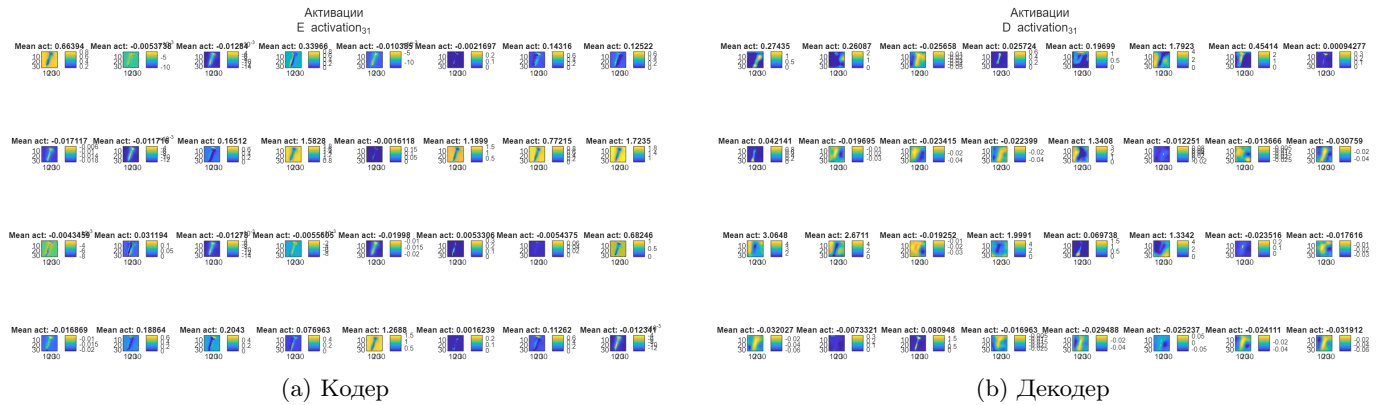


Рис. 10: Активации третьего слоя Кодера и третьего слоя декодера

2 Задачи для дальнейшей работы

На текущем этапе проверка работы модели на "дефектных" данных невозможна в силу малой точности реконструкции входных изображений.

1. Изучение влияния значений гиперпараметров на работу модели
2. Изучение связи между признаками объектов на изображениях и формой латентного пространства
3. Повышение точности работы модели
4. Распирение модели до Variational Autoencoder